

Github Account Set up:

1. **Configure Git on their computer**
2. **Create a GitHub repository and connect it to Git Bash**

They only need to do the Git configuration **once** on each computer.

Part 1 — Configure Git Bash

Open **Git Bash** and have each student run:

1. Set their name

They should use the name they want associated with their Git commits:

```
git config --global user.name "John Doe"
```

Example:

```
git config --global user.name "Chigozie Dennis"
```

2. Set their GitHub email

Use the email address associated with their GitHub account:

```
git config --global user.email "your-email@example.com"
```

For example:

```
git config --global user.email "student@gmail.com"
```

3. Verify the configuration

```
git config --global --list
```

They should see:

```
user.name=John Doe  
user.email=student@gmail.com
```

Part 2 — Create their first repository

Tell the students to go to GitHub and click:

New repository

For their first practice project, they could name it:

```
git-practice
```

Choose:

- **Public** — if it's a portfolio/practice project
- **Add README:** No
- **Add .gitignore:** No
- **License:** None

Then click **Create repository**.

Part 3 — Create a local project

On their Windows computer, create a folder, for example:

```
Desktop  
└─ git-practice
```

Inside that folder, create a simple file:

```
hello.txt
```

Put something inside it, such as:

```
My first GitHub project.
```

Part 4 — Open Git Bash in that folder

Right-click inside the `git-practice` folder and choose:

Open Git Bash here

Then run:

```
pwd
```

and:

```
ls
```

They should see:

```
hello.txt
```

Part 5 — Initialize Git

Run:

```
git init
```

This turns the ordinary folder into a Git repository.

They should see something similar to:

```
Initialized empty Git repository
```

Part 6 — Check Git status

```
git status
```

They should see `hello.txt` listed as an untracked file.

Part 7 — Add the file

```
git add .
```

Then:

```
git status
```

Now `hello.txt` should appear under **Changes to be committed**.

Part 8 — Make their first commit

```
git commit -m "Initial commit"
```

This saves a version of their project locally.

Part 9 — Rename the branch to main

```
git branch -M main
```

Part 10 — Connect Git Bash to GitHub

Go back to the GitHub repository they created.

GitHub will provide a repository URL similar to:

```
https://github.com/STUDENT-USERNAME/git-practice.git
```

They should run:

```
git remote add origin https://github.com/STUDENT-USERNAME/git-practice.git
```

For example:

```
git remote add origin https://github.com/john-doe/git-practice.git
```

Verify the connection:

```
git remote -v
```

They should see:

```
origin  https://github.com/john-doe/git-practice.git (fetch)
origin  https://github.com/john-doe/git-practice.git (push)
```

Part 11 — Push to GitHub

Finally:

```
git push -u origin main
```

After authentication, their `hello.txt` should appear on GitHub.

The complete beginner cheat sheet

I would give your students this exact sequence:

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
git config --global --list
cd path/to/your/project
git init
git status
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/YOUR-REPOSITORY.git
git remote -v
git push -u origin main
```

Git and GitHub are not the same thing.

Git
↓
Runs on their computer
↓
Tracks project changes
↓
git add → git commit
↓
GitHub
↓
Stores/shares the repository online
↓
git push

So the key workflow they should eventually memorize is:

git add → git commit → git push

And when they clone an existing GitHub project, the workflow changes slightly to:

git clone → edit → git add → git commit → git push